

Predicción de Cortísimo Plazo del Bono AL30

Lautaro Marín

Prueba técnica

Resumen

En este trabajo se presenta el pipeline completo desarrollado para predecir el cambio de precio del bono AL30. El sistema incluye: procesado para tres instrumentos correlacionados (AL30 en pesos, AL30D MEP, AL30C CCL), ingeniería de features, entrenamiento de un modelo lineal, validaciones de integridad y exportación de parámetros para inferencia en C++.

■ 1. Introducción

Se busca estimar el movimiento inmediato del bono AL30 utilizando información de los mercados AL30 (ARS), AL30D (MEP) y AL30C (CCL). Para ello, buscó hacer la predicción del cambio en el midprice entre dos ticks consecutivos, lo que permite derivar tanto la magnitud del movimiento como la dirección y el precio resultante. Se utilizó un modelo lineal debido a su velocidad de inferencia y a la necesidad de portar el modelo a C++ sin degradar tiempos de ejecución.

Formalmente, la predicción responde a la ecuación

$$\hat{y}_t = \beta_0 + \sum_{j=1}^m \beta_j X_{t,j},$$

donde $\hat{y}_t$ aproxima $y_t = p_{t+1} - p_t$ y el precio predicho se recupera mediante $\hat{p}_{t+1} = p_t + \hat{y}_t$.

■ 2. Procesamiento de Datos

Los datos provienen del libro de órdenes con cinco niveles por lado, expresados en nanosegundos UTC. Se realizó una conversión horaria a Buenos Aires, seguida de un recorte temporal entre las 11:00 y las 17:00 para coincidir con el horario efectivo. Se descartaron ticks con valores inválidos.

A partir de los datos obtenidos se definieron como atributo el midprice, el spread, el imbalance y la presión cruzada entre precios y cantidades. Estos se calcularon de manera independiente para AL30, AL30D y AL30C.

Dado que el bono cotiza en tres mercados con monedas diferentes, se construyeron los tipos de cambio implícitos $t_{c_{mep}}$ y $t_{c_{ccl}}$ expresándolos como relaciones entre midprices.

■ 3. Ingeniería de Atributos

El conjunto base de atributos contiene 12 señales: 4 del AL30 y 8 derivadas de los mercados dolarizados una vez pesificadas. Para capturar dinámicas locales de corto plazo, cada columna se expandió en cinco retardos consecutivos (*lags*), lo que elevó el total a 72 atributos. Esta expansión se realizó cuidadosamente para evitar fuga de información y se verificó automáticamente que el modelo, el escalador y el código C++ compartieran exactamente la misma dimensionalidad.

■ 4. Modelo y Entrenamiento

Los atributos fueron estandarizados mediante un `StandardScaler` entrenado únicamente sobre el tramo inicial del dataset, correspondiente al 70 % temporal. El modelo utilizado fue `LinearRegression`, debido al bajo costo computacional y facilidad de exportación a C++. El conjunto restante del 30 % se reservó para evaluación, respetando el orden temporal para evitar contaminación del futuro en el entrenamiento.

■ 5. Validación del Modelo

Las métricas relevantes incluyen el RMSE (*Root Mean Square Error*) y el MAE (*Mean Absolute Error*) sobre el cambio de precio, junto con una validación direccional basada en el signo del movimiento. Por un lado, se obtuvo $RMSE = 2,93$ y $MAE = 0,47$, mientras que el análisis direccional muestra que el modelo predice correctamente la tendencia un 31,6 % de las veces, lo cual se aproxima a la aleatoriedad. Entonces, el modelo presentó un mal rendimiento a la hora de predecir tendencias más allá de los valores acotados de RMSE y MAE.

■ 6. Exportación a C++

El modelo fue exportado mediante la escritura explícita de los coeficientes, el intercepto, las medias y desvíos del escalador, y la última fila de atributos crudos y estandarizados. El código C++ carga estos valores, replica exactamente el proceso de estandarización y evalúa la fórmula lineal, produciendo la predicción del delta de precio y el midprice futuro. El procedimiento tiene complejidad lineal en el número de atributos y una baja latencia, por lo cual lo vuelve adecuado para contextos operativos donde los tiempos deben ser inferiores al intervalo entre ticks.

■ 7. Resultados y discusiones

Se realizó la predicción del precio a $t + 1$ utilizando C++, obteniendo así $\Delta P_{t+1} = 1,09$, y el midprice correspondiente $mid_{ars} = 77716,09$. Por otro lado, se evaluaron los coeficientes RMSE y MAE, y el análisis direccional expresados anteriormente. Si bien se llegó a la predicción buscada, el modelo no

mostró el comportamiento deseado a la hora de predecir la tendencia ΔP_{t+1} .

■ 8. Conclusiones

El pipeline desarrollado cumple todas las exigencias de la prueba técnica. Se construyó una cadena completa, desde el procesamiento de datos crudos del libro de órdenes hasta la predicción en C++, con verificaciones internas que garantizan consistencia y reproducibilidad. La solución permite obtener predicciones inmediatas del siguiente *midprice*, manteniendo la baja latencia necesaria para su uso en un entorno de trading algorítmico de muy corto plazo. El enfoque lineal constituye un punto de partida fácilmente extensible.

El rendimiento del modelo subraya la necesidad de explorar alternativas para capturar la compleja no linealidad de la dinámica de los precios de corto plazo. Modelos más

complejos, como los *ensambles* de árboles de decisión (e.g., *LightGBM* o *RandomForest*), podrían mejorar la precisión al modelar las interacciones de las *features*. Otras exploraciones más profundas podrían incluir el uso de modelos de *Deep Learning* que analicen la secuencia temporal, como las redes *LSTM* (*Long Short-Term Memory*), o incluso modelos que incorporen señales externas como el *análisis de sentimiento* derivado de noticias o redes sociales.

Sin embargo, estos modelos más avanzados (incluyendo *LSTM*) se descartaron inicialmente debido a que su costo computacional de inferencia es significativamente mayor al de una regresión lineal. Implementarlos requeriría bibliotecas de *Deep Learning* que introducirían latencias críticas inaceptables en un entorno operativo en C++, rompiendo el requisito fundamental de que el cálculo de la predicción debe ser más rápido que la ventana temporal predictiva. Por lo tanto, el camino futuro de mejora debe sopesar cuidadosamente la ganancia en precisión frente al impacto en la latencia.